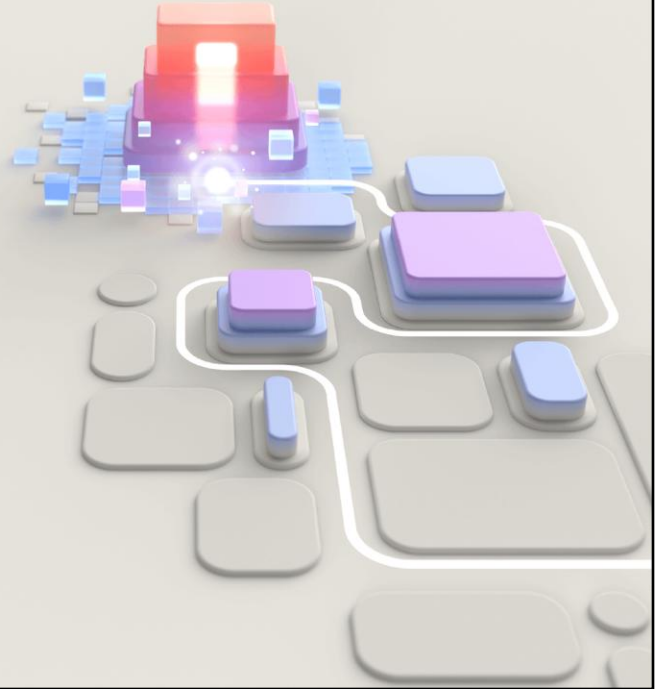




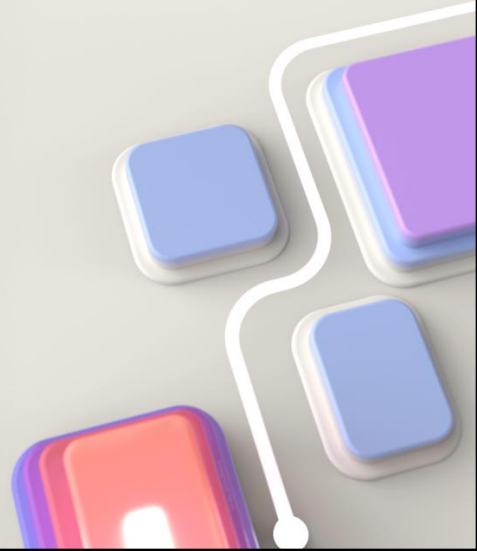
PL-400T00 Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.



Extending Microsoft Dataverse

© Copyright Microsoft Corporation. All rights reserved.



Modules



- Introduction to Dataverse for developers
- Create plug-ins

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Introduction to Dataverse for developers
2. Create plug-ins

Module 1: Introduction to Dataverse for developers

© Copyright Microsoft Corporation. All rights reserved.

In this module, you will:

- Explain what functions can be executed against Microsoft Power Platform via Microsoft Power Platform SDKs.
- Perform basic operations against Microsoft Power Platform such as create/read/update/delete operations.

Learning Objectives

After completing this module, you will be able to:

- 1** Explain what functions can be executed against Microsoft Power Platform via Microsoft Power Platform SDKs.
- 2** Perform basic operations against Microsoft Power Platform such as create/read/update/delete operations.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Introduction to Dataverse for developers

<https://learn.microsoft.com/training/modules/intro-cds-developers-power-platform>



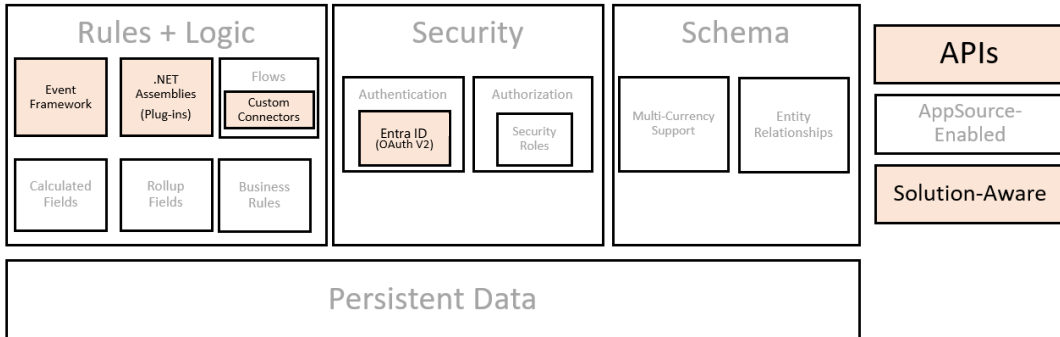
© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

Introduction to Dataverse for developers

<https://learn.microsoft.com/training/modules/intro-cds-developers-power-platform>

Dataverse extensibility model



To better understand how to extend Dataverse, it is valuable to learn about its underlying architecture.

© Copyright Microsoft Corporation. All rights reserved.

To better understand how to extend the Microsoft Dataverse, it is valuable to learn about its underlying architecture. Since the Dataverse is a software-as-a-service platform, most of these details such as underlying data storage, are successfully abstracted from us as developers and we can focus on more interesting items such as building custom business logic and integrating with other applications. The below graphic illustrates these underlying components, highlighted sections are areas where which model-driven Power App developers can extend leveraging code.

[!NOTE] This unit is only intended to be a high-level view of the Dataverse architecture. For more in-depth analysis of the Dataverse, refer to the [Dataverse Developer Guide](#).

The Dataverse provides a metadata-driven architecture to provide flexibility to create custom tables and extend existing tables. By using this approach, we are then able to easily transport our customizations across environments via a declarative solution that we can maintain as zip files or extract into XML files when applying the Microsoft's Solution Packager. Solution Packager is the preferred tool to compress and extract a solution file so you can check it into source control repository. For more information, see [Use the SolutionPackager tool to compress and extract a solution file](#).

When you create a custom table, metadata about that table, its attributes and relationships to other tables, and user interface components (such as forms and views) are stored as metadata in the system.

The Dataverse also exposes an OData V4 RESTful endpoint, known as the Web API, that can be leveraged to view and manipulate metadata within a Dataverse instance. One common mechanism for browsing your solution's metadata is to leverage the Metadata Browser. Microsoft provides this as a solution that you can install into your app. The current links are found under the developer documentation under [Microsoft Docs](#). This tool can be installed into any model-driven Power App built on the Dataverse. In addition, [XrmToolbox](#) has a community metadata browser tool available.

Plug-ins are managed code classes that are registered to run when specific events occur. When the event occurs, they pass through a plug-in execution pipeline that consists of three stages: PreValidation, PreOperation, and PostOperation. When registering a plug-in, you must choose which stage in which your code should run. The stage you should choose depends on the purpose of the extension. You don't need to apply all your business logic within a single step.

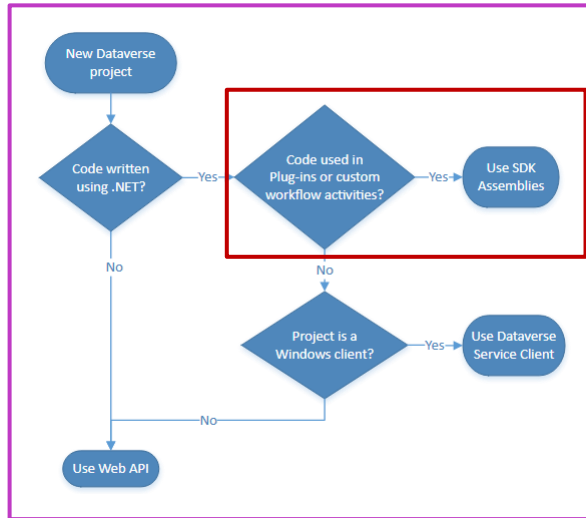
Plug-ins can also be configured run either synchronously or asynchronously. Synchronous plugins execute immediately according to the stage and execution order, and the entire operation will wait until they complete. Asynchronous plug-ins are executed via a system job after the operation completes, and as such can only be registered for the PostOperation stage.

Plug-ins can also be developed and registered as Custom Workflow Activities. Custom Workflow Activities are a tool that allows developers to produce reusable components that can be leveraged by an administrator to conduct actions not provided by the out-of-the-box workflow editor.

The web services provide a programmatic interface for applications and services to access business data, organization information, and metadata stored in Dataverse. The main service to access business data is called the Organization Service. There are two protocols available for application code to access this service, each with its own web endpoint: OData and SOAP. A RESTful Web API provides access to the OData V4 protocol of the web service while an SDK provides a library of classes and methods to access to the SOAP endpoint. The Web API is the more modern and preferred endpoint for new application code, while the SOAP endpoint is more for legacy applications and plug-in/custom workflow activity development. Both the Web API and the SOAP endpoints are functionally equivalent, except that the SOAP endpoint must be used for the development of plug-ins and custom workflow activities.

.NET Assemblies (plug-ins)

Plug-ins are managed code classes that are registered to run when specific events occur.



© Copyright Microsoft Corporation. All rights reserved.

Web services provide a programmatic interface for applications and services to access business data, organization information, and metadata that is stored in Dataverse. The main service to access business data is called the Organization Service. Two protocols are available for application code to access this service, each with its own web endpoint: ODATA and SOAP. A RESTful Web API provides access to the ODATA V4 protocol of the web service, while an SDK provides a library of classes and methods to access the SOAP endpoint.

The Web API is the more modern and preferred endpoint for new application code, while the SOAP endpoint is more for legacy applications and plug-in and/or custom workflow activity development. Both the Web API and the SOAP endpoints are functionally equivalent, except that the SOAP endpoint must be used for the development of plug-ins and custom workflow activities.

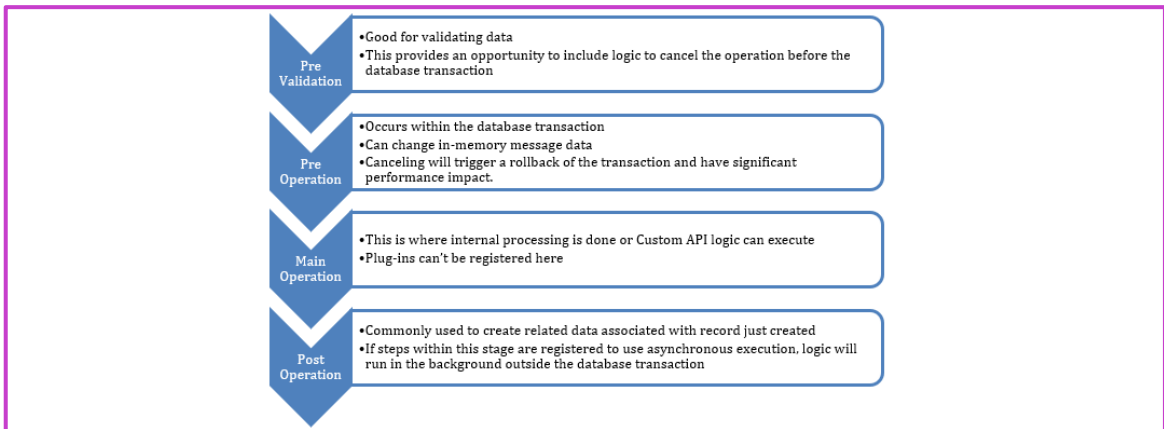
Plug-ins are managed code classes that are registered to run when specific events occur. When the event occurs, it passes through a plug-in pipeline that consists of three stages: PreValidation, PreOperation, and PostOperation. When registering a plug-in, you must choose which stage in which your code should run. The stage that you should choose depends on the purpose of the extension. You don't need to apply all your business logic within a single step.

Plug-ins can also be configured to run either synchronously or asynchronously. Synchronous plug-ins run immediately according to the stage and run order, and the entire operation will wait until they complete. Asynchronous plug-ins are run through a system job after the operation completes and can only be registered for the PostOperation stage.

Plug-ins can also be developed and registered as custom workflow activities. Custom workflow activities is a tool that allows developers to produce reusable components that can be used by an administrator to perform actions not provided by the out-of-the-box workflow editor.

Event pipeline

When you perform an action like create a record in an app or create a record using the API, a Create message is processed by Dataverse.



© Copyright Microsoft Corporation. All rights reserved.

When you perform an action like create a record in an app or create a record using the API, a Create message is processed by Dataverse. The message is processed in the event pipeline that provides a consistent set of stages that the message goes through. Each stage except for the main operation can have a plugin attached to it to run custom logic. The following are the stages supported as a message starts at the top and progresses through the stages of the pipeline:

Having a strong understanding of how messages are processed can help you understand behaviors, and how and where best to implement custom logic. Understanding the pipeline is also essential to recognizing how plugins and custom APIs fit in.

The **PreValidation** event occurs first in the pipeline, before any security checks are performed. It is intended for usage to ensure that the user executing the current transaction has the correct permissions needed to perform the intended operation.

As a developer, you can leverage this event to run validation logic and cancel the operation before the transaction occurs. For example, if configured to run whenever a table is updated, you have the ability to cancel the operation before the update occurs by throwing an `InvalidPluginExecutionException` method within your plugin's execution logic. For more information on Execution context, see [Understand the Execution Context](#).

PreOperation Use this event if you want to change any values of the table prior to it being saved.

Post Operation Use this event to modify any properties of the message before it is returned to the caller. Make sure not to apply any updates to the corresponding table at this point as it will trigger another update event!

<https://learn.microsoft.com/power-apps/developer/data-platform/event-framework>

Execution modes (Synchronous versus Asynchronous)

Plug-ins can be configured to run synchronously or asynchronously, depending on what type of pipeline operation we are handling.

Plug-ins registered in synchronous mode will run as soon as their execution pipeline stage is reached, and the entire operation will not proceed until the logic has completed execution.

Plug-ins registered in asynchronous mode will be dispatched as a system job to the Asynchronous service, which executes their logic after the given operation completes.

© Copyright Microsoft Corporation. All rights reserved.

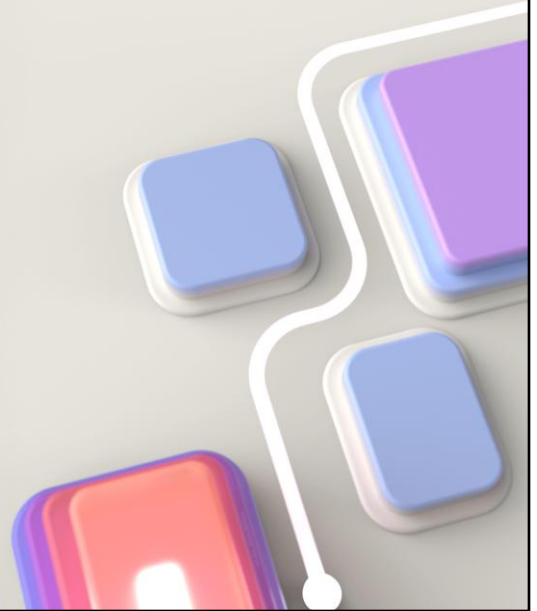
Plug-ins can be configured to run synchronously or asynchronously, depending on what type of pipeline operation we are handling.

Plug-ins registered in synchronous mode will run as soon as their execution pipeline stage is reached, and the entire operation will not proceed until the logic has completed execution. If multiple plug-ins are registered to run against the same pipeline stage, the Execution Order column (specified via the Plugin Registration Tool) will determine which one runs first.

Plug-ins registered in asynchronous mode will be dispatched as a system job to the Asynchronous service, which executes their logic after the given operation completes. For more information on how system jobs work, see [Asynchronous service](#). Asynchronous plug-ins can only be registered for the **PostOperation** stage of the Event Pipeline.

Module 2: Create plug-ins

© Copyright Microsoft Corporation. All rights reserved.



Learning Objectives

After completing this module, you will be able to:

- 1 Build plug-ins
- 2 Debug plug-ins

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Extend plug-ins

<https://learn.microsoft.com/training/modules/plug-ins-extend-power-platform>



© Copyright Microsoft Corporation. All rights reserved.

Link to student courseware on Learn

Extend plug-ins

<https://learn.microsoft.com/training/modules/plug-ins-extend-power-platform>

Introduction to Plug-ins

A plug-in is *imperative* logic that should be used only when a *declarative* process, such as a business rule, flow, or workflow, does not meet your requirement.

- Canceling the event and displaying an error to the user.
- Making changes to the data in the operation.
- Initiating other actions by using the Organization Service to add automation.

© Copyright Microsoft Corporation. All rights reserved.

A plug-in is *imperative* logic that should be used only when a *declarative* process, such as a business rule, flow, or workflow, does not meet your requirement.

Fundamentally, a plug-in is merely a .NET assembly that implements an IPlugin interface, which can be found in the Microsoft.CrmSdk.CoreAssemblies NuGet package. The IPlugin interface exposes a single method, Execute, which is where you can place any custom logic that you want to invoke based on whatever event you are handling.

Common scenarios of when to use plug-ins are:

- Canceling the event and displaying an error to the user.
- Making changes to the data in the operation.
- Initiating other actions by using the Organization Service to add automation.

Alternatives to plug-ins

Plug-ins should be considered a last resort in many cases.

- Dataverse workflows
- Power Automate cloud flows
- Calculated and rollup columns
- Custom actions

© Copyright Microsoft Corporation. All rights reserved.

Plug-ins should be considered a last resort in many cases. Although plug-ins are powerful and, if well written, highly performant, it's important to minimize the amount of custom/imperative logic that you place into your system, because it can affect maintainability, upgradability, and so on.

Common alternatives to plug-ins are:

- Dataverse classic and real-time workflows
- Power Automate
- Calculated and Rollup Columns
- Custom Actions

Plug-in considerations

- Plug-ins perform better when you consider their performance, capabilities, and ability to run synchronously.
- A well-written plug-in will always be the most efficient way to apply business logic to Dataverse.
- Plug-ins provide several capabilities that are not available with declarative business logic, such as efficiently working with external services in code.
- If synchronous logic is required for your application, plug-ins might be a required option for you.

© Copyright Microsoft Corporation. All rights reserved.

Plug-ins perform better when you consider their performance, capabilities, and ability to run synchronously.

Performance: A well-written plug-in will always be the most efficient way to apply business logic to Dataverse. However, a poorly written plug-in can create a significant negative impact on the performance of your environment.

Capabilities: Plug-ins provide several capabilities that are not available with declarative business logic, such as efficiently working with external services in code. Nevertheless, Power Automate is rapidly approaching parity with plug-ins.

Ability to run synchronously: If synchronous logic is required for your application, plug-ins might be a required option for you. However, on-demand workflows can also run synchronously and should be considered, depending on your requirements.

Workflows/flows vs. plug-ins/client script

Circumstances might occur when existing limitations require you to develop plug-ins to accomplish certain activities.

	Workflow	Plug-in	Client Script
Synchronous	Either	Either	Synchronous
Access External Data	No	Yes	Yes (with limitations)
Maintenance	Business Users	Developers	Developers
Can Run As	User	Any licensed user or current user	User
Can Run On Demand	Yes	No	No
Can Nest Child Processes	Yes	Yes	No
Execution Stage	Before/After	Before/After	Before/After
Triggers	Create, Column Change, Status Change, Assign to Owner, On Demand	Create, Column Change, Status Change, Assign to Owner, Delete, along with many other specialized triggers	Column Change or Form Load

© Copyright Microsoft Corporation. All rights reserved.

Building plugins

Plugins are .NET classes that implement a **IPlugin** interface provided by the Dataverse SDK assemblies.

- **IPluginExecutionContext** – This gives you access to the message being processed and information about the requestor.
- **ITracingService** – This gives you access to write to the trace log for diagnostic purposes.
- **IOrganizationServiceFactory** – This gives you access to retrieve an **OrganizationService** for use in accessing data from the plug-in.

© Copyright Microsoft Corporation. All rights reserved.

Plugins are .NET classes that implement a **IPlugin** interface provided by the Dataverse SDK assemblies. That interface requires you implement only one method named **Execute**. The following is an example of a minimal implementation:

```
public sealed class MyFirstPlugin : IPlugin { public void Execute(IServiceProvider serviceProvider) { //Business Logic } }
```

The **Execute** method has one parameter of type **IServiceProvider**. This interface has one method **GetService** that you can use to retrieve services available to a plugin. The following are examples of the services available:

IPluginExecutionContext - this gives you access to the message being processed and information about the requestor.

ITracingService - this gives you access to write to the trace log for diagnostic purposes.

IOrganizationServiceFactory - this gives you access to retrieve an **OrganizationService** for use in accessing data from the plug-in.

The following is an example of using **GetService** to get the execution context:

```
IPluginExecutionContext context = (IPluginExecutionContext) serviceProvider.GetService(typeof(IPluginExecutionContext));
```

Using **context.InputParameters** can give you access to the originating message and **context.OutputParameters** can give you access to what is being returned to the caller.

A good understanding of the execution context is fundamental for a plugin developer. You can learn more about the execution context [here](#).

To have a plugin execute, it must be registered to run for a specific message. This can be accomplished using the Plugin Registration tool.

Plug-in execution context

Event Execution pipeline passes information about the current operation in the `IPluginExecutionContext` interface to the plug-in:

- `PrimaryEntityId`
- `PrimaryEntityName`
- `MessageName`
- `Stage`
- `Depth`
- `InputParameters`
- `OutputParameters`
- `PreEntityImages`
- `PostEntityImages`

© Copyright Microsoft Corporation. All rights reserved.

```
IPluginExecutionContext context =  
  
    (IPluginExecutionContext)serviceProvider  
        .GetService(typeof(IPluginExecutionConte  
xt));  
  
var entity =  
    (Entity)context.InputParameters["Target"];
```

Whenever a plug-in (or custom workflow extension) runs, a wealth of data is made available by Dataverse that contains information about the context in which the current operation resides.

In plug-ins, the *IPluginExecutionContext* is accessible through the *IServiceProvider* parameter of the *Execute* method by calling the *GetService* method.

<https://learn.microsoft.com/power-apps/developer/data-platform/understand-the-data-context>

<https://learn.microsoft.com/dotnet/api/microsoft.xrm.sdk.ipuginexecutioncontext>

Develop a plug-in

Inherit from IPlugin interface

Execute method

Extract execution content

Access Organization service

Impersonation

- context.UserId
- context.InitiatingUserId

© Copyright Microsoft Corporation. All rights reserved.

```
public class MyPlugin : IPlugin
{
    public void Execute(IServiceProvider
serviceProvider)
    {
        IPluginExecutionContext context =
        IPluginExecutionContext)serviceProvide
r.GetService(typeof(IPluginExecutionCo
ntext));
        IOrganizationServiceFactory
serviceFactory =
        (IOrganizationServiceFactory)servicePr
ovider.GetService(typeof(IOrganization
ServiceFactory));
        IOrganizationService orgService =
        serviceFactory.CreateOrganizationServi
ce(context.UserId);
    }
}
```

<https://learn.microsoft.com/power-apps/developer/data-platform/plugin-ins>

<https://learn.microsoft.com/power-apps/developer/data-platform/write-plugin-in>

<https://learn.microsoft.com/power-apps/developer/data-platform/impersonate-a-user>

Error handling

Organization service errors
generate ServiceFaults

To show a message to the user
create an
InvalidPluginExecutionException

```
try { }  
catch  
(FaultException<OrganizationServiceFault>  
ex)  
{  
    throw new  
        InvalidPluginExecutionException("The  
        following error occurred in MyPlugin.",  
        ex);  
}  
catch (Exception ex)  
{  
    tracingService.Trace("MyPlugin: error:  
    {0}", ex.ToString());  
    throw;  
}
```

© Copyright Microsoft Corporation. All rights reserved.

Tracing service

Tracing Service

Enable trace logging

- Off
- Exceptions
- All

```
ITracingService tracingService =  
(ITracingService)serviceProvider.GetService  
(typeof(ITracingService));
```

```
tracingService.Trace("Write {0} {1}",  
"your", "message");
```

© Copyright Microsoft Corporation. All rights reserved.

Develop a plug-in

PluginBase abstract class

Power Platform Tools for Visual Studio

- Power Platform Plug-in Library template

Power Platform CLI

Run CLI command
`pac plugin init`

© Copyright Microsoft Corporation. All rights reserved.

Demo: Build a plug-in

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Prevalidation plugin that checks for a missing column and throws a new `InvalidPluginExecutionException`

InputParameters

Input parameters are contained in the execution context's InputParameters collection and are of type *Entity*.

- Register your plug-in to run on the PreValidation stage on Create and/or Update of the table that you want to validate.
- Validate data in your plug-in by reading the values from the InputParameters collection. On Create, you'll want to retrieve the Target collection.
- Throw an InvalidPluginExecutionException if the provided data is not valid.

© Copyright Microsoft Corporation. All rights reserved.

Input parameters are contained in the execution context's InputParameters collection and are of type *Table*. This property enables you to see what table values were presented to the plug-in prior to the implementation of a given operation.

For example, if you want to validate your data and prevent it from being saved if validation is not successful, the steps that you take would be similar to the following:

1. Register your plug-in to run on the PreValidation stage on Create and/or Update of the table that you want to validate.
1. Validate data in your plug-in by reading the values from the InputParameters collection. On Create, you'll want to retrieve the Target collection.
1. Throw an InvalidPluginExecutionException if the provided data is not valid.

Or, perhaps you want to update values on your table (such as stripping special characters from a phone number) before the data is saved. You might handle this situation with the following steps:

1. Register your plug-in to run on the PreOperation stage on Create and/or Update of the table that you want to update.
1. Update the data in your plug-in by editing the values from the InputParameters collection. On Create, you'll want to retrieve the Target collection.

OutputParameters

Output parameters are contained in the execution context's OutputParameters collection and are of the type *Table*.

- Register your plug-in to run on the PostOperation stage on Create and/or Update of the table that you want to update.
- Update the data in your plug-in by editing the values from the OutputParameters collection. Note that on Create, you'll want to retrieve the Target collection.

© Copyright Microsoft Corporation. All rights reserved.

Output parameters are contained in the execution context's OutputParameters collection and are of the type *Table*. Output parameters are only provided after the given operation has occurred. As such, you'll only be able to use this collection when you are handling events in the PostOperation stage.

For example, if you want to change the values that are returned by the operation, you can modify them by conducting the following steps:

1. Register your plug-in to run on the PostOperation stage on Create and/or Update of the table that you want to update.
1. Update the data in your plug-in by editing the values from the OutputParameters collection. Note that on Create, you'll want to retrieve the Target collection.

Pre and Post Entity Images

- Allow plug-in code to reference columns not in the operation
- Before (Pre) and after (Post) values
- Prevent the need to retrieve a row as first step in plug-in

© Copyright Microsoft Corporation. All rights reserved.

When registering plug-ins to run against the event framework, you can provide an immutable copy, or snapshot, of the data. This is ideal, especially when you need to reference values of the data before or after an operation has occurred, for example, for logging or custom auditing purposes.

While you can retrieve table values by using the Organization Context from within a plug-in by means of a Retrieve request, using table images to conduct this task is much more efficient and highly advised.

<https://docs.microsoft.com/en-us/powerapps/developer/common-data-service/register-plugin#register-plugin-step>

Debug and troubleshoot a plug-in

- ITracing service
- Raise InvalidPluginExecution Exception to display message to user
- Plug-in Trace logs
- System jobs for asynchronous plug-ins
- Plug-in profiler
- Analyze plug-in performance with Analytics in the Power Platform admin center
- Timeout: 2 minute limit
- Error handling

© Copyright Microsoft Corporation. All rights reserved.

Write to the tracing service inside of a plug-in
Enable plug-in trace logging
Debug plug-in using plug-in profiler
Replay plug-in execution
Run the plug-in profiler standalone
Analyze plug-in performance with Analytics in
Power Platform Admin center

<https://docs.microsoft.com/powerapps/developer/common-data-service/logging-tracing>

<https://docs.microsoft.com/powerapps/developer/common-data-service/debug-plug-in>

<https://docs.microsoft.com/powerapps/developer/common-data-service/troubleshoot-plug-in>

[https://docs.microsoft.com/powerapps/developer](https://docs.microsoft.com/powerapps/developer/common-data-service/troubleshoot-plug-in)

[oper/common-data-service/analyze-performance](https://docs.microsoft.com/powerapps/developer/common-data-service/tutorial-debug-plugin-in)

<https://docs.microsoft.com/powerapps/developer/common-data-service/tutorial-debug-plugin-in>

Optimize plug-in performance

- Filter the columns to trigger plug-in so only run when required
- Use Pre and Post image to remove need to retrieve row as first step
- Do not register plug-in on any table
- Minimize operations as 2 minute limit
- Check the depth of the transaction and abort if necessary
- Handle all errors

© Copyright Microsoft Corporation. All rights reserved.

Do not retrieve row as first step, Use of Any
Table, Depth
2 minute limit
Error handling

<https://docs.microsoft.com/en-us/powerapps/developer/common-data-service/best-practices/business-logic/>

It's talked about later, but in practice in projects, we tend to go down the Azure aware plug-in route involving a Function when we hit bottlenecks with server-side plug-ins.

Configure a Custom API message

Create own messages in Dataverse APIs

Tables

- Custom API
- Custom API Request Parameter
- Custom API Response Property

Global or Bound to a table

Create with

- Power Apps maker portal
- Plugin Registration Tool
- Code

© Copyright Microsoft Corporation. All rights reserved.

Create a new message for the Dataverse API

Is Function

- Function vs Action
- GET vs POST
- Function returns data

Is Private

- Hides from metadata service calls

<https://learn.microsoft.com/power-apps/developer/data-platform/custom-actions>

<https://learn.microsoft.com/power-apps/developer/data-platform/custom-api>

<https://learn.microsoft.com/power-apps/developer/data-platform/create-custom-api-maker-portal>

Develop a plug-in that implements a Custom API

- Add plug-in to perform logic when custom API is called
- Write code just like you would any other plug-in
- Register the assembly
- Select the assembly Custom API Plugin Type in the Custom API record
- Do not need to register a step

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/data-platform/custom-api#write-a-plug-in-for-your-custom-api>

Learning Path 6 practice labs



Extending Microsoft Dataverse

Lab 8: Dataverse Plug-ins

© Copyright Microsoft Corporation. All rights reserved.

In this lab you will build two plug-ins. The first plug-in will run when a new permit record is created, and it will check that there are no other permits that exists for the build sites that are *locked*. If a locked permit is found, the plug-in will block the creation of this new permit. The second plug-in will hook up and run when the Lock Permit custom API is invoked. In the prior module we defined the custom API, and now with the plug-in step registered on execution of the custom API, it will perform the lock permit business logic.

Summary



- Plug-ins are a powerful method that you have to extend business logic within Microsoft Dataverse
- Plug-ins interact with the event pipeline and provide an execution context that enables you to handle scenarios that otherwise might not have been accomplished through declarative means

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Introduction to Dataverse for developers
2. Create plug-ins



© Copyright Microsoft Corporation. All rights reserved.